Models and databases

Konu Başlıkları :

Database transactions
Multiple databases
Tablespaces

Hazırlayan : Aydın Yiğit Oğuz

3.2.7 Veritabanı işlemleri

Django, veritabanı işlemlerinin nasıl yönetileceğini kontrol etmeniz için birkaç yol sunar.

Veritabanı işlemlerini yönetme Django'nun varsayılan işlem davranışı

Django'nun varsayılan davranışı, otomatik onay modunda çalışmaktır. Aktif bir işlem olmadığı sürece, her sorgu hemen veritabanına kaydedilir. Ayrıntılar için aşağıya bakın.

Django, birden fazla sorgu gerektiren ORM işlemlerinin, özellikle de delete() ve update() sorgularının bütünlüğünü garanti etmek için işlemleri veya kaydetme noktalarını otomatik olarak kullanır.

Django'nun *TestCase* sınıfı da performans nedenleriyle her testi bir işlem içine alır.

İşlemleri HTTP isteklerine bağlama

Web'de işlemleri yönetmenin yaygın bir yolu, her isteği bir işlem içine almaktır. Bu davranışı etkinleştirmek istediğiniz her veritabanının yapılandırmasında *ATOMIC_REQUESTS* değerini `True` olarak ayarlayın.

Bu şu şekilde çalışır. Bir görünüm işlevini çağırmadan önce, Django bir işlem başlatır. Yanıt sorunsuz bir şekilde üretilirse, Django işlemi tamamlar. Görünüm bir istisna üretirse, Django işlemi geri alır.

Görünüm kodunuzda, genellikle *atomic()* bağlam yöneticisiyle, kaydetme noktalarını kullanarak alt işlemler gerçekleştirebilirsiniz. Ancak, görünümün sonunda değişikliklerin ya tamamı ya da hiçbiri onaylanacaktır.

> **Uyarı**
>
> Bu işlem modelinin sadeliği çekici olsa da, trafik arttığında verimsiz hale gelir. Her görüntüleme için bir işlem açmak, bir miktar ek yük getirir. Performans üzerindeki etkisi, uygulamanızın sorgu kalıplarına ve veritabanınızın kilitlemeyi ne kadar iyi yönettiğine bağlıdır.

> **İstek başına işlemler ve akış yanıtları**
>
> Bir görünüm *StreamingHttpResponse* döndürdüğünde, yanıtın içeriğini okumak genellikle içeriği oluşturmak için kodu çalıştırır. Görünüm zaten döndüğü için, bu tür kod işlem dışında çalışır.

Genel olarak, akış yanıtı oluştururken veritabanına yazmak tavsiye edilmez, çünkü yanıtı göndermeye başladıktan sonra hataları yönetmenin mantıklı bir yolu yoktur.

Uygulamada, bu özellik her görünüm işlevini aşağıda açıklanan *atomic()* dekoratörüne sarar.

Yalnızca görünümünüzün yürütülmesinin işlemlerin içinde yer aldığını unutmayın. Orta katman yazılımı işlemin dışında çalışır ve şablon yanıtlarının oluşturulması da öyle.

ATOMIC_REQUESTS etkinleştirildiğinde bile, görünümlerin bir işlem içinde çalışmasını engellemek mümkündür.

non_atomic_requests(using=None)

Bu dekoratör, belirli bir görünüm için *ATOMIC_REQUESTS*'in etkisini geçersiz kılar:

```
from django.db import transaction



@transaction.non_atomic_requests
def my_view(request): do_stuff()



@transaction.non_atomic_requests(using="other")
def my_other_view(request):
    do_stuff_on_the_other_database()
```

Bu, yalnızca görünümün kendisine uygulandığında çalışır.

İşlemleri açıkça kontrol etme

Django, veritabanı işlemlerini yönetmek için tek bir API sunar.

atomic(using=None, savepoint=True, durable=False)

Atomiklik, veritabanı işlemlerinin belirleyici özelliğidir. `atomic`, veritabanında atomikliğin garanti edildiği bir kod bloğu oluşturmamızı sağlar. Kod bloğu başarıyla tamamlanırsa, değişiklikler veritabanına kaydedilir. Bir istisna oluşursa, değişiklikler geri alınır.

`atomic` blokları iç içe yerleştirilebilir. Bu durumda, iç blok başarıyla tamamlandığında, daha sonra dış blokta bir istisna ortaya çıkarsa, iç bloğun etkileri yine de geri alınabilir.

Bazen, bir `atomik` bloğun her zaman en dıştaki `atomik` blok olmasını sağlamak yararlı olabilir; bu sayede, bloktan hata olmadan çıkıldığında veritabanındaki tüm değişikliklerin kaydedilmesi garanti altına alınır. Bu, "dayanıklılık" olarak bilinir ve `durable=True` ayarıyla sağlanabilir. `Atomik` blok başka bir bloğun içine yerleştirilmişse, bir `RuntimeError hatası` verilir.

`atomic`, hem dekoratör olarak kullanılabilir:

```
from django.db import transaction



@transaction.atomic
def viewfunc(request):
    # Bu kod bir işlem içinde çalıştırılır.
    do_stuff()
```

ve bir bağlam yöneticisi olarak:

```
from django.db import transaction



def viewfunc(request):
    # Bu kod otomatik onay modunda (Django'nun varsayılanı) çalışır.
    do_stuff()

    with transaction.atomic():
        # Bu kod bir işlem içinde çalışır.
        do_more_stuff()
```

`Atomic'i` bir try/except bloğuna sarmak, bütünlük hatalarının doğal bir şekilde işlenmesini sağlar:

```
from django.db import IntegrityError, transaction



@transaction.atomic
def viewfunc(request): create_parent()

    try:
        with transaction.atomic():
            generate_relationships()
    except IntegrityError:
        handle_exception()

    add_children()
```

Bu örnekte, `generate_relationships() işlevi` bir bütünlük kısıtlamasını ihlal ederek bir veritabanı hatasına yol açsa bile, `add_children()` işlevinde sorgular çalıştırabilirsiniz; ayrıca `create_parent()` işlevinden kaynaklanan değişiklikler hâlâ mevcut olacak ve aynı işleme bağlı kalacaktır. `handle_exception()` işlevi çağrıldığında, `generate_relationships()` işlevinde gerçekleştirilmeye çalışılan tüm işlemlerin çoktan güvenli bir şekilde geri alınmış olacağına dikkat edin

çağrıldığında güvenli bir şekilde geri alınmış olacağını unutmayın; bu sayede istisna işleyicisi de gerekirse veritabanı üzerinde işlem yapabilir.

`Atomik işlemlerin` içinde istisnaları yakalamaktan kaçının!

`Atomik` bir bloktan çıkılırken, Django işlemin normal bir şekilde mi yoksa bir istisna ile mi sonlandırıldığını kontrol ederek, işlemin onaylanıp onaylanmayacağını veya geri alınacağını belirler. `Atomik` bir blok içinde istisnaları yakalayıp işlerseniz, bir sorun oluştuğunu Django'dan gizleyebilirsiniz. Bu durum beklenmedik davranışlara yol açabilir.

Bu durum, çoğunlukla *DatabaseError* ve *IntegrityError* gibi alt sınıfları için geçerlidir. Böyle bir hatadan sonra işlem kesilir ve Django, `atomik` bloğun sonunda geri alma işlemi gerçekleştirir. Geri alma işlemi gerçekleşmeden önce veritabanı sorguları çalıştırmaya çalışırsanız, Django bir *TransactionManagementError hatası* verir. ORM ile ilgili bir sinyal işleyicisi bir istisna oluşturduğunda da bu davranışla karşılaşabilirsiniz.

Veritabanı hatalarını yakalamanın doğru yolu, yukarıda gösterildiği gibi bir `atomik` blok içinde yapmaktır. Gerekirse, bu amaçla ek bir `atomik` blok ekleyin. Bu yaklaşımın bir başka avantajı da, bir istisna oluştuğunda hangi işlemlerin geri alınacağının açıkça belirlenmiş olmasıdır.

Ham SQL sorguları tarafından ortaya çıkan istisnaları yakalarsanız, Django'nun davranışı belirsizdir ve veritabanına bağlıdır.

Bir işlemi geri alırken uygulama durumunu manuel olarak geri yüklemeniz gerekebilir.

Bir işlem geri alınması durumunda, modelin alanlarının değerleri eski haline dönmez. Orijinal alan değerlerini manuel olarak geri yüklemediğiniz sürece bu durum, modelin tutarsız bir duruma yol açabilir.

Örneğin, `aktif` bir alana sahip `MyModel` modelinde, bu kod parçacığı, işlemde active değerini True olarak güncelleme başarısız olursa, sonunda yer alan `if obj.active` kontrolünün, işlemde `active` değerinin `True` olarak güncellenmesi başarısız olursa doğru değeri kullanmasını sağlar:

```
from django.db import DatabaseError, transaction

obj = MyModel(active=False)
obj.active = True
try:
    with transaction.atomic(): obj.save()
except DatabaseError:
    obj.active = False

if obj.active:
    ...
```

Bu durum, önbellekleme veya küresel değişkenler gibi uygulama durumunu tutabilecek diğer tüm mekanizmalar için de geçerlidir.

Örneğin, kod bir nesneyi kaydettikten sonra önleyici olarak önbellekteki verileri güncelliyorsa, önbellek değişikliklerini işlem gerçekten tamamlanana kadar ertelemek için bunun yerine transaction.on_commit() kullanılması önerilir.

Atomikliği garanti altına almak için, `atomic` bazı API'leri devre dışı bırakır. Bir `atomic` bloğu içinde veritabanı bağlantısının autocommit durumunu onaylamaya, geri almaya veya değiştirmeye çalışmak bir istisna oluşturacaktır.

`atomic,` bir veritabanının adı olması gereken bir `using` argümanı alır. Bu argüman sağlanmazsa, Django `"default"` veritabanını kullanır.

Arka planda, Django'nun işlem yönetimi kodu:

- en dıştaki `atomik` bloğa girildiğinde bir işlem açar;
- iç `atomik` bloğa girildiğinde bir kaydetme noktası oluşturur;
- iç bloktan çıkıldığında işlem tamamlanır veya kaydetme noktasına geri alınır;
- en dıştaki bloktan çıkarken işlemi onaylar veya geri alır.

`savepoint` argümanını `False` olarak ayarlayarak iç bloklar için kaydetme noktalarının oluşturulmasını devre dışı bırakabilirsiniz. Bir istisna meydana gelirse, Django, varsa bir kaydetme noktası olan ilk üst bloktan çıkarken, yoksa en dıştaki bloktan çıkarken geri alma işlemini gerçekleştirir. Atomiklik yine de dış işlem tarafından garanti edilir. Bu seçenek yalnızca kaydetme noktalarının getirdiği ek yükün fark edilebilir olduğu durumlarda kullanılmalıdır. Yukarıda açıklanan hata işlemeyi bozması bir dezavantajdır.

Autocommit kapalıyken `atomic seçeneğini` kullanabilirsiniz. Bu seçenek, en dıştaki blok için bile yalnızca kaydetme noktalarını kullanır.

Performansla ilgili hususlar

Açık işlemler, veritabanı sunucunuz için bir performans yükü oluşturur. Bu yükü en aza indirmek için işlemlerinizi olabildiğince kısa tutun. Bu durum, Django'nun istek/yanıt döngüsü dışında, uzun süren işlemlerde *atomic() işlevini* kullanıyorsanız özellikle önemlidir.

Otomatik onay

Django neden otomatik onay kullanır?

SQL standartlarında, halihazırda aktif bir işlem yoksa her SQL sorgusu bir işlem başlatır. Bu tür işlemler daha sonra açıkça onaylanmalı veya geri alınmalıdır.

Bu, uygulama geliştiricileri için her zaman uygun değildir. Bu sorunu hafifletmek için çoğu veritabanı bir otomatik onaylama modu sunar. Otomatik onaylama açıkken ve aktif bir işlem yoksa, her SQL sorgusu kendi işlemi içinde sarılır. Başka bir deyişle, bu tür her sorgu bir işlem başlatmakla kalmaz, aynı zamanda sorgunun başarılı olup olmadığına bağlı olarak işlem otomatik olarak onaylanır veya geri alınır.

PEP 249, Python Veritabanı API Spesifikasyonu v2.0, otomatik onaylamanın başlangıçta kapalı olmasını gerektirir. Django bu varsayılan ayarı geçersiz kılar ve otomatik onaylamayı açar.

Bunu önlemek için işlem yönetimini devre dışı bırakabilirsiniz, ancak bu önerilmez.

İşlem yönetimini devre dışı bırakma

Belirli bir veritabanı için Django'nun işlem yönetimini, yapılandırmasında *AUTOCOMMIT değerini* `False` olarak ayarlayarak tamamen devre dışı bırakabilirsiniz. Bunu yaparsanız, Django otomatik onaylamayı etkinleştirmez ve hiçbir onay işlemi gerçekleştirmez. Altta yatan veritabanı kütüphanesinin normal davranışını görürsünüz.

Bu, Django veya üçüncü taraf kütüphaneler tarafından başlatılanlar da dahil olmak üzere her işlemi açıkça onaylamanızı gerektirir. Bu nedenle, bu seçenek en çok kendi işlem kontrol orta katman yazılımınızı çalıştırmak veya gerçekten sıra dışı bir şey yapmak istediğiniz durumlarda kullanılır.

Commit işleminden sonra eylemler gerçekleştirme

Bazen, mevcut veritabanı işlemiyle ilgili bir eylem gerçekleştirmeniz gerekir, ancak bu yalnızca işlem başarıyla kaydedildiğinde geçerlidir. Örnekler arasında arka plan görevi, e-posta bildirimi veya önbellek geçersiz kılma sayılabilir.

on_commit(), açık işlemin başarıyla tamamlanmasının ardından yürütülecek geri çağırmaları kaydetmenize olanak tanır:

on_commit(func, using=None, robust=False)

on_commit() işlevine bir işlev veya herhangi bir

çağrılabilir nesne aktarın:

```
from django.db import transaction


def send_welcome_email(): ...


transaction.on_commit(send_welcome_email)
```

Geri çağrılara herhangi bir argüman geçilmez, ancak bunları `functools.partial()` ile bağlayabilirsiniz:

```
from functools import partial

for user in users: transaction.on_commit(partial(send_invite_email,
    user=user))
```

Geri çağırma işlevleri, açık işlem başarıyla kaydedildikten sonra çağrılır. İşlem bunun yerine geri alınırsa (genellikle bir *atomic()* bloğunda işlenmemiş bir istisna oluştuğunda), geri çağırma işlevi geçersiz hale gelir ve hiçbir zaman çağrılmaz.

Açık bir işlem yokken `on_commit() işlevini` çağırırsanız, geri çağırma işlevi hemen yürütülür.

Bazen başarısız olabilecek geri çağırmaları kaydetmek yararlı olabilir. `robust=True parametresini` geçirmek, mevcut geri çağırma bir istisna atsa bile sonraki geri çağırmaların yürütülmesini sağlar. Python'un `Exception` sınıfından türetilen tüm hatalar yakalanır ve `django.db.backends.base` logger'a kaydedilir.

on_commit() ile kaydedilen geri çağırmaları test etmek için *TestCase.captureOnCommitCallbacks()* işlevini kullanabilirsiniz.

Kaydetme noktaları

Kaydetme noktaları (yani iç içe geçmiş *atomic()* blokları) doğru şekilde işlenir. Yani, bir kaydetme noktasından sonra (iç içe geçmiş bir *atomic()* bloğu içinde) kaydedilen bir *on_commit()* işlevi, dış işlem tamamlandıktan sonra çağrılır; ancak işlem sırasında o kaydetme noktasına veya daha önceki herhangi bir kaydetme noktasına geri dönüş (rollback) gerçekleşirse çağrılmaz:

```
with transaction.atomic(): # Dış atomik, yeni bir işlem başlat
    transaction.on_commit(foo)

    with transaction.atomic(): # İç atomik blok, bir kaydetme noktası oluştur
        transaction.on_commit(bar)

# En dıştaki bloktan çıkıldığında foo() ve ardından bar() çağrılacaktır
```

Öte yandan, bir kaydetme noktası geri alınırsa (bir istisna atılması nedeniyle), iç çağrılabilir işlev çağrılmayacaktır:

```
with transaction.atomic(): # Dış atomik, yeni bir işlem başlat
    transaction.on_commit(foo)

    try:
        transaction.atomic() ile: # İç atomik blok, bir kaydetme noktası oluştur
            transaction.on_commit(bar)
            raise SomeError() # İstisna oluşturma - kaydetme noktasını iptal et
    except SomeError:
        pass

# foo() çağrılacak, ancak bar() çağrılmayacak
```

Yürütme sırası

Belirli bir işlem için on-commit işlevleri, kaydedildikleri sırayla yürütülür.

İstisna işleme

Belirli bir işlem içinde `robust=False` ile kaydedilmiş bir on-commit işlevi yakalanmamış bir istisna oluşturursa, aynı işlemde daha sonra kaydedilmiş işlevler çalıştırılmaz. Bu, *on_commit()* kullanmadan işlevleri kendiniz sırayla çalıştırmış olmanızla aynı davranıştır.

Yürütme zamanlaması

Geri çağırmalarınız başarılı bir commit işleminden sonra yürütülür; bu nedenle bir geri çağırmada meydana gelen bir hata, işlemin geri alınmasına neden olmaz. Bunlar, işlemin başarısına bağlı olarak koşullu olarak yürütülür, ancak işlemin bir parçası değildir. Hedeflenen kullanım senaryoları (e-posta bildirimleri, arka plan görevleri vb.) için bu durum uygun olmalıdır. Eğer değilse (eğer takip eyleminiz o kadar kritikse ki, başarısızlığı işlemin kendisinin başarısızlığı anlamına geliyorsa), o zaman *on_commit()* kancasını kullanmak istemezsiniz. Bunun yerine, psycopg İki Aşamalı İşlem protokol desteği ve Python DB-API spesifikasyonundaki isteğe bağlı İki Aşamalı İşlem Uzantıları gibi iki aşamalı işlemleri kullanmak isteyebilirsiniz.

Geri çağırma işlevleri, commit işleminden sonra bağlantıda otomatik işleme moduna geri dönülene kadar çalıştırılmaz (çünkü aksi takdirde geri çağırma işlevinde yapılan herhangi bir sorgu, örtük bir işlem açarak bağlantının otomatik işleme moduna geri dönmesini engeller).

Otomatik onay modundayken ve *atomic()* bloğunun dışındayken, işlev onay sırasında değil, hemen çalışır.

On-commit işlevleri yalnızca otomatik onay modunda ve *atomic()* (veya *ATOMIC_REQUESTS*) işlem API'si ile çalışır. Otomatik onay devre dışı bırakıldığında ve bir atomic bloğu içinde olmadığınızda *on_commit() işlevini* çağırmak bir hataya neden olur.

Testlerde kullanım

Django'nun *TestCase* sınıfı, testlerin birbirinden izole olmasını sağlamak amacıyla her testi bir işlem içine alır ve her testin ardından bu işlemi geri alır. Bu, hiçbir işlemin aslında kaydedilmediği anlamına gelir; dolayısıyla *on_commit()* geri çağırma işlevleriniz asla çalıştırılmaz.

TestCase.captureOnCommitCallbacks() kullanarak bu sınırlamayı aşabilirsiniz. Bu, *on_commit()* geri çağırmalarınızı bir listeye yakalar ve bunlara ilişkin doğrulamalar yapmanıza veya bunları çağırarak işlemin onaylanmasını taklit etmenize olanak tanır.

Bu sınırlamayı aşmanın bir başka yolu da *TestCase* yerine *TransactionTestCase* kullanmaktır. Bu, işlemlerinizin onaylandığı ve geri çağırmaların çalışacağı anlamına gelir. Ancak *TransactionTestCase*, testler arasında veritabanını boşaltır ve bu, TestCase'in izolasyonundan önemli ölçüde daha yavaştır.

Neden geri alma kancası yok?

Bir geri alma kancasını, bir onay kancasına kıyasla sağlam bir şekilde uygulamak daha zordur; zira birçok farklı durum, örtük bir geri almaya neden olabilir.

Örneğin, işleminiz düzgün bir şekilde kapatılmadan sonlandırıldığı için veritabanı bağlantınız kesilirse, rollback hook'unuz asla çalışmaz.

Ancak bir çözüm var: atomik blok (işlem) sırasında bir şey yapıp işlem başarısız olursa bunu geri almak yerine, *on_commit()* işlevini kullanarak işlemi, işlem başarılı olana kadar erteleyin. Hiç yapmadığınız bir şeyi geri almak çok daha kolaydır!

Düşük seviyeli API'ler

> **Uyarı**
>
> Mümkünse her zaman *atomic()* işlevini tercih edin. Bu işlev, her veritabanının kendine özgü özelliklerini dikkate alır ve geçersiz işlemleri önler.
>
> Düşük seviyeli API'ler, yalnızca kendi işlem yönetiminizi uyguluyorsanız yararlıdır.

Otomatik onay

Django, her veritabanı bağlantısının otomatik onay durumunu yönetmek için *django.db.transaction* modülünde bir API sağlar.

get_autocommit(using=None)

set_autocommit(autocommit, using=None)

Bu işlevler, bir veritabanının adı olması gereken bir `using` argümanı alır. Bu argüman belirtilmezse, Django `"default"` veritabanını kullanır.

Otomatik onaylama başlangıçta açıktır. Eğer kapatırsanız, geri yüklemek sizin sorumluluğunuzdadır.

Otomatik onaylamayı kapattığınızda, veritabanı adaptörünüzün varsayılan davranışını alırsınız ve Django size yardımcı olmaz. Bu davranış PEP 249'da belirtilmiş olsa da, adaptörlerin uygulamaları her zaman birbiriyle tutarlı değildir. Kullandığınız adaptörün belgelerini dikkatlice inceleyin.

Otomatik onaylamayı yeniden etkinleştirmeden önce, genellikle *commit()* veya *rollback()* komutlarını vererek hiçbir işlemin etkin olmadığından emin olmalısınız.

Django, *atomic()* bloğu etkin olduğunda otomatik onaylamayı kapatmayı reddeder, çünkü bu atomikliği bozar.

İşlemler

İşlem, veritabanı sorgularından oluşan atomik bir kümedir. Programınız çökse bile, veritabanı tüm değişikliklerin uygulanacağını veya hiçbirinin uygulanmayacağını garanti eder.

Django, işlem başlatmak için bir API sağlamaz. İşlemi başlatmanın beklenen yolu, *set_autocommit()* ile otomatik onaylamayı devre dışı bırakmaktır.

Bir işlem içindeyken, o ana kadar yaptığınız değişiklikleri *commit()* ile kaydetmeyi ya da *rollback()* ile iptal etmeyi seçebilirsiniz. Bu işlevler *django.db.transaction* modülünde tanımlanmıştır.

commit(using=None) **rollback**(using=None)

Bu işlevler, bir veritabanının adı olması gereken `using` argümanını alır. Bu argüman sağlanmazsa, Django `"default"` veritabanını kullanır.

Django, *atomic()* bloğu etkin olduğunda commit veya rollback işlemlerini reddeder, çünkü bu durum atomikliği bozar.

Kaydetme noktaları

Bir kaydetme noktası, işlemin tamamını değil, işlemin bir kısmını geri almanızı sağlayan, işlem içindeki bir işaretçidir. Kaydetme noktaları SQLite, PostgreSQL, Oracle ve MySQL (InnoDB depolama motoru kullanıldığında) arka uçlarında kullanılabilir. Diğer arka uçlar da kaydetme noktası işlevlerini sunar, ancak bunlar boş işlemlerdir; aslında hiçbir şey yapmazlar.

Django'nun varsayılan davranışı olan otomatik onay (autocommit) kullanıyorsanız, kaydetme noktaları pek kullanışlı değildir. Ancak, *atomic()* ile bir işlem açtığınızda, onay veya geri alma işlemini bekleyen bir dizi veritabanı işlemi oluştururusunuz. Geri alma işlemi gerçekleştirirseniz, işlemin tamamı geri alınır. Kaydetme noktaları, `transaction.rollback()` ile gerçekleştirilecek tam geri alma yerine, daha ayrıntılı bir geri alma işlemi gerçekleştirme olanağı sağlar.

atomic() dekoratörü iç içe geçtiğinde, kısmi commit veya rollback'e izin vermek için bir savepoint oluşturur. Aşağıda açıklanan işlevler yerine *atomic()* kullanmanız şiddetle tavsiye edilir, ancak bunlar hala genel API'nin bir parçasıdır ve kullanımdan kaldırılmaları planlanmamaktadır.

Bu işlevlerin her biri, davranışın geçerli olacağı veritabanının adını içeren bir `` `using` `` argümanı alır. Eğer `` `using` `` argümanı belirtilmezse, `"varsayılan"` veritabanı kullanılır.

Kaydetme noktaları, *django.db.transaction* içindeki üç işlev tarafından kontrol edilir:

savepoint(using=None)

Yeni bir kaydetme noktası oluşturur. Bu, işlemin içinde "iyi" durumda olduğu bilinen bir noktayı işaretler. Kaydetme noktası kimliğini (`sid`) döndürür.

savepoint_commit(sid, using=None)

savepoint `sid'i` serbest bırakır. Savepoint oluşturulduğundan beri yapılan değişiklikler işlemin bir parçası haline gelir.

savepoint_rollback(sid, using=None)

İşlemi `sid` kaydetme noktasına geri alır.

Kaydetme noktaları desteklenmiyorsa veya veritabanı otomatik onay modundaysa bu işlevler hiçbir şey

yapmaz. Ayrıca, bir yardımcı işlev de vardır:

clean_savepoints(using=None)

Benzersiz kaydetme noktası kimliklerini oluşturmak için kullanılan sayacı sıfırlar.

Aşağıdaki örnek, kaydetme noktalarının kullanımını göstermektedir:

```
from django.db import transaction



# işlem aç
@transaction.atomic def
viewfunc(request):
    a.save()
    # işlem artık a.save()'i içerir

    sid = transaction.savepoint()

    b.save()
    # işlem artık a.save() ve b.save()'i içeriyor

    if want_to_keep_b: transaction.savepoint_commit(sid)
        # açık işlem hala a.save() ve b.save()'i içeriyor
    else:
        transaction.savepoint_rollback(sid))
        # açık işlem artık sadece a.save() içeriyor
```

Kayıt noktaları, kısmi geri alma işlemi gerçekleştirerek veritabanı hatasından kurtulmak için kullanılabilir. Bunu *atomic()* bloğu içinde yaparsanız, durumun daha alt bir düzeyde halledildiğini bilmediği için bloğun tamamı geri alınacaktır! Bunu önlemek için, aşağıdaki işlevlerle geri alma davranışını kontrol edebilirsiniz.

get_rollback(using=None)

set_rollback(rollback, using=None)

Geri alma bayrağını `True` olarak ayarlamak, en içteki atomik bloktan çıkılırken geri almayı zorlar. Bu, istisna oluşturmadan geri almayı tetiklemek için yararlı olabilir.

Bunu `False` olarak ayarlamak, böyle bir geri almayı engeller. Bunu yapmadan önce, işlemi mevcut atomik blok içindeki bilinen iyi bir kaydetme noktasına geri aldığınızdan emin olun! Aksi takdirde, atomikliği bozarsınız ve veri bozulması meydana gelebilir.

Veritabanına özgü notlar SQLite'daki kaydetme noktaları

SQLite kaydetme noktalarını desteklemesine rağmen, sqlite3 modülünün tasarımındaki bir kusur nedeniyle bunların kullanımı neredeyse imkansızdır.

Autocommit etkinleştirildiğinde, kaydetme noktaları anlamsız hale gelir. Devre dışı bırakıldığında, `sqlite3` kaydetme noktası deyimlerinden önce örtük olarak işlemleri tamamlar. (Aslında, `SELECT`, `INSERT`, `UPDATE`, `DELETE` ve `REPLACE` dışındaki tüm deyimlerden önce işlemleri tamamlar.) Bu hatanın iki sonucu vardır:

- Kayıt noktaları için düşük seviyeli API'ler yalnızca bir işlem içinde, yani bir *atomic()* bloğu içinde kullanılabilir.
- Autocommit kapalıyken *atomic()* kullanmak imkansızdır.

MySQL'deki İşlemler

MySQL kullanıyorsanız, tablolarınız işlemleri destekleyebilir veya desteklemeyebilir; bu, MySQL sürümünüze ve kullandığınız tablo türlerine bağlıdır. ("Tablo türleri" derken, "InnoDB" veya "MyISAM" gibi türleri kastediyoruz.) MySQL işlemlerinin özellikleri bu makalenin kapsamı dışındadır, ancak MySQL sitesinde MySQL işlemleri hakkında bilgiler bulunmaktadır.

MySQL kurulumunuz işlemleri desteklemiyorsa, Django her zaman otomatik onay modunda çalışır: komutlar çağrıldıkları anda yürütülür ve onaylanır. MySQL kurulumunuz işlemleri destekliyorsa, Django bu belgede açıklandığı şekilde işlemleri yönetir.

PostgreSQL işlemlerinde istisnaların işlenmesi

> **Not**
>
> Bu bölüm, yalnızca kendi işlem yönetiminizi uyguluyorsanız geçerlidir. Bu sorun, Django'nun varsayılan modunda ortaya çıkmaz ve *atomic()* bunu otomatik olarak yönetir.

Bir işlem içinde, PostgreSQL imlecine yapılan bir çağrı bir istisna (genellikle `IntegrityError`) oluşturduğunda, aynı işlemdeki sonraki tüm SQL sorguları "mevcut işlem iptal edildi, sorgular işlem bloğunun sonuna kadar yok sayıldı" hatasıyla başarısız olur. `save()` işlevinin temel kullanımı PostgreSQL'de bir istisna oluşturma olasılığı düşük olsa da, benzersiz alanlara sahip nesneleri kaydetme, `force_insert/force_update` bayrağını kullanarak kaydetme veya özel SQL çağrısı yapma gibi daha gelişmiş kullanım modelleri istisna oluşturabilir.

Bu tür hatalardan kurtulmanın birkaç yolu vardır.

İşlem geri alma

İlk seçenek, işlemin tamamını geri almaktır. Örneğin:

```
a.save() # Başarılı olur, ancak işlem geri alınmasıyla iptal edilebilir
try:
    b.save() # İstisna oluşturabilir
except IntegrityError:
    transaction.rollback()
c.save() # Başarılı, ancak a.save() geri alınmış olabilir
```

`transaction.rollback()` işlevini çağırmak, işlemin tamamını geri alır. Onaylanmamış tüm veritabanı işlemleri kaybolur. Bu örnekte, `a.save()` tarafından yapılan değişiklikler, bu işlemin kendisi herhangi bir hata oluşturmamış olsa bile kaybolacaktır.

Kaydetme noktası geri alma

Geri alma işleminin kapsamını kontrol etmek için kaydetme noktalarını kullanabilirsiniz. Başarısız olabilecek bir veritabanı işlemi gerçekleştirmeden önce, kaydetme noktasını ayarlayabilir veya güncelleyebilirsiniz; bu şekilde, işlem başarısız olursa, tüm işlemi geri almak yerine sadece sorun yaratan tek bir işlemi geri alabilirsiniz. Örneğin:

```
a.save() # Başarılı olur ve kaydetme noktası geri alma ile asla geri alınmaz
sid = transaction.savepoint()
try:
    b.save() # İstisna oluşturabilir
    transaction.savepoint_commit(sid)
except IntegrityError: transaction.savepoint_rollback(sid)
c.save() # Başarılı olur ve a.save() asla geri alınmaz
```

Bu örnekte, `b.save()` bir istisna oluşturursa `a.save()` geri alınmaz.

3.2.8 Birden fazla veritabanı

Bu konu kılavuzu, Django'nun birden fazla veritabanıyla etkileşim için sunduğu desteği açıklamaktadır. Django belgelerinin geri kalanının çoğu, tek bir veritabanıyla etkileşim kurduğunuzu varsayar. Birden fazla veritabanıyla etkileşim kurmak istiyorsanız, bazı ek adımlar atmanız gerekecektir.

Ayrıca

Birden fazla veritabanıyla test etme hakkında bilgi için Çoklu veritabanı desteğine bakın.

Veritabanlarınızı tanımlama

Django ile birden fazla veritabanı kullanmanın ilk adımı, Django'ya kullanacağınız veritabanı sunucularını bildirmektir. Bu, *DATABASES* ayarı kullanılarak yapılır. Bu ayar, Django genelinde belirli bir veritabanına başvurmanın bir yolu olan veritabanı takma adlarını, o bağlantıya ait bir ayar sözlüğüne eşler. İç sözlüklerdeki ayarlar, *DATABASES* belgelerinde ayrıntılı olarak açıklanmaktadır.

Veritabanlarına istediğiniz herhangi bir takma ad verebilirsiniz. Ancak, `"default"` takma adı özel bir öneme sahiptir. Django, başka bir veritabanı seçilmediğinde `"default"` takma adına sahip veritabanını kullanır.

Aşağıda, iki veritabanını tanımlayan bir `settings.py` örneği yer almaktadır – bir varsayılan PostgreSQL veritabanı ve `users` adlı bir MySQL veritabanı:

```
DATABASES = {
    "default": {
        "NAME": "app_data",
        "ENGINE": "django.db.backends.postgresql",
        "USER": "postgres_user",
        "PASSWORD": "s3krit",
    },
    "users": {
        "NAME": "user_data",
        "ENGINE": "django.db.backends.mysql", "USER":
        "mysql_user",
        "PASSWORD": "priv4te",
    },
}
```

`Varsayılan` veritabanı kavramı projenizin bağlamında anlamlı gelmiyorsa, kullanmak istediğiniz veritabanını her zaman açıkça belirtmeye özen göstermelisiniz. Django, bir `varsayılan` veritabanı girişinin tanımlanmasını gerektirir; ancak bu giriş kullanılmayacaksa, parametreler sözlüğü boş bırakılabilir. Bunu yapmak için, kullandığınız tüm katkılar ve üçüncü taraf uygulamalar dahil olmak üzere tüm uygulamalarınızın modelleri için *DATABASE_ROUTERS'ı* ayarlamanız gerekir, böylece hiçbir sorgu varsayılan veritabanına yönlendirilmez. Aşağıda, `varsayılan` girişin kasıtlı olarak boş bırakıldığı, iki varsayılan olmayan veritabanını tanımlayan bir `settings.py` örneği bulunmaktadır:

```
DATABASES = {
    "default": {},
    "users": {
        "NAME": "user_data",
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
        "ENGINE": "django.db.backends.mysql",
        "USER": "mysql_user",
        "PASSWORD": "superS3cret",
    },
    "customers": {
        "NAME": "customer_data",
        "ENGINE": "django.db.backends.mysql",
        "USER": "mysql_cust",
        "PASSWORD": "veryPriv@ate",
    },

}
```

DATABASES ayarınızda tanımlamadığınız bir veritabanına erişmeye çalışırsanız, Django bir `django.utils.connection.ConnectionDoesNotExist` istisnası oluşturacaktır.

Veritabanlarınızı senkronize etme

Migrate yönetim komutu, her seferinde tek bir veritabanı üzerinde çalışır. Varsayılan olarak, `varsayılan` veritabanı üzerinde çalışır, ancak *--database* seçeneğini kullanarak farklı bir veritabanını senkronize etmesini sağlayabilirsiniz. Dolayısıyla, yukarıdaki ilk örnekte tüm modelleri tüm veritabanlarına senkronize etmek için şunu çağırmanız gerekir:

```
$ ./manage.py migrate
$ ./manage.py migrate --database=users
```

Her uygulamanın belirli bir veritabanıyla senkronize edilmesini istemiyorsanız, belirli modellerin kullanılabilirliğini kısıtlayan bir kural uygulayan bir veritabanı yönlendiricisi tanımlayabilirsiniz.

Yukarıdaki ikinci örnekte olduğu gibi, `varsayılan` veritabanını boş bıraktıysanız, *migrate komutunu* her çalıştırdığınızda bir veritabanı adı belirtmeniz gerekir. Veritabanı adını belirtmezseniz bir hata oluşur. İkinci örnek için:

```
$ ./manage.py migrate --database=users
$ ./manage.py migrate --database=customers
```

Diğer yönetim komutlarını kullanma

Veritabanıyla etkileşime giren diğer `django-admin` komutlarının çoğu, *migrate* ile aynı şekilde çalışır – her seferinde yalnızca bir veritabanı üzerinde çalışır ve `--database seçeneğini` kullanarak kullanılan veritabanını kontrol eder.

Bu kuralın bir istisnası *makemigrations* komutudur. Bu komut, yeni göçler oluşturmadan önce mevcut göç dosyalarındaki sorunları (dosyaların düzenlenmesi nedeniyle ortaya çıkmış olabilecek) tespit etmek için veritabanlarındaki göç geçmişini doğrular. Varsayılan olarak, yalnızca `varsayılan` veritabanını kontrol eder, ancak yüklü ise yönlendiricilerin *allow_migrate()* yöntemine başvurur.

Otomatik veritabanı yönlendirme

Birden fazla veritabanını kullanmanın en kolay yolu, bir veritabanı yönlendirme şeması kurmaktır. Varsayılan yönlendirme şeması, nesnelerin orijinal veritabanlarına "bağlı" kalmasını sağlar (yani, `foo` veritabanından alınan bir nesne aynı veritabanına kaydedilir). Varsayılan yönlendirme şeması, bir veritabanı belirtilmediğinde tüm sorguların `varsayılan` veritabanına yönlendirilmesini sağlar.

Varsayılan yönlendirme şemasını etkinleştirmek için hiçbir şey yapmanız gerekmez; bu şema her Django projesinde "kullanıma hazır" olarak sağlanır. Ancak, daha ilginç veritabanı tahsis davranışları uygulamak istiyorsanız, kendi veritabanı yönlendiricilerinizi tanımlayıp yükleyebilirsiniz.

Veritabanı yönlendiricileri

Bir veritabanı yönlendiricisi, en fazla dört yöntem sunan bir sınıftır:

db_for_read(model, **hints)

`model` türündeki nesneler için okuma işlemleri sırasında kullanılacak veritabanını önerir.

Bir veritabanı işlemi, veritabanı seçimine yardımcı olabilecek herhangi bir ek bilgi sağlayabiliyorsa, bu bilgi `hints` sözlüğünde sağlanır. Geçerli ipuçları hakkında ayrıntılar aşağıda verilmiştir.

Öneri yoksa `None değerini` döndürür.

db_for_write(model, **hints)

Model türündeki nesnelerin yazma işlemleri için kullanılacak veritabanını önerir.

Bir veritabanı işlemi, veritabanı seçiminde yardımcı olabilecek herhangi bir ek bilgi sağlayabiliyorsa, bu bilgi `ipuçları` sözlüğünde sunulur. Geçerli ipuçlarıyla ilgili ayrıntılar aşağıda verilmiştir.

Öneri yoksa `None değerini` döndürür.

allow_relation(obj1, obj2, **hints)

`obj1` ve `obj2` arasındaki ilişkiye izin verilmesi gerekiyorsa `True`, ilişkinin engellenmesi gerekiyorsa `False`, yönlendiricinin bir görüşü yoksa `None değerini` döndürür. Bu, tamamen bir doğrulama işlemidir ve yabancı anahtar ve çoktan çoka işlemlerinde iki nesne arasındaki ilişkiye izin verilip verilmeyeceğini belirlemek için kullanılır.

Hiçbir yönlendiricinin bir görüşü yoksa (yani tüm yönlendiriciler `None` döndürürse), yalnızca aynı veritabanı içindeki ilişkiler izin verilir.

allow_migrate(db, app_label, model_name=None, **hints)

`"db"` takma adıyla anılan veritabanında taşıma işleminin çalıştırılmasına izin verilip verilmediğini belirler. İşlemin çalıştırılması gerekiyorsa `True`, çalıştırılmaması gerekiyorsa `False`, yönlendiricinin bir görüşü yoksa `None değerini` döndürür.

`app_label` konumsal argümanı, taşınan uygulamanın etiketidir.

`model_name`, çoğu taşıma işlemi tarafından taşınan modelin `model._meta.model_name` (model `adının` küçük harfli versiyonu) değerine ayarlanır. *RunPython* ve *RunSQL* işlemleri için, ipuçları kullanılarak sağlanmadıkça değeri `None'dir`.

`hints,` belirli işlemler tarafından yönlendiriciye ek bilgiler iletmek için kullanılır.

`model_name` ayarlandığında, `hints` normalde `'model'` anahtarı altında model sınıfını içerir. Bunun geçmiş bir model olabileceğini ve dolayısıyla herhangi bir özel öznitelik, yöntem veya görünümsahip olmayabileceğini unutmayın. Yalnızca `_meta'ya` güvenmelisiniz.

Bu yöntem, belirli bir veritabanında bir modelin kullanılabilirliğini belirlemek için de kullanılabilir.

makemigrations, model değişiklikleri için her zaman göç dosyaları oluşturur; ancak `allow_migrate()` `işlevi` `False` `değerini` döndürürse, `veritabanında` *migrate komutu* çalıştırıldığında `model_name` için tüm göç işlemleri hiçbir uyarı vermeden atlanır. Halihazırda göç dosyaları bulunan modellerde `allow_migrate()` işlevinin davranışını değiştirmek, yabancı anahtarların bozulmasına, fazladan tabloların oluşmasına veya tabloların kaybolmasına neden olabilir. *makemigrations*, göç geçmişini doğrularken, hiçbir uygulamanın göç yapmasına izin verilmeyen veritabanlarını atlar.

Bir yönlendirici tüm bu yöntemleri sağlamak zorunda değildir; bunlardan bir veya birkaçını atlayabilir. Yöntemlerden biri atlanırsa, Django ilgili kontrolü gerçekleştirirken o yönlendiriciyi atlar.

İpuçları

Veritabanı yönlendiricisi tarafından alınan ipuçları, belirli bir isteğin hangi veritabanına yönlendirileceğine karar vermek için kullanılabilir.

Şu anda sağlanacak tek ipucu, devam eden okuma veya yazma işlemiyle ilişkili bir nesne örneği olacaktır. Bu, kaydedilmekte olan örnek olabilir ya da çok-çok ilişkisinde eklenmekte olan bir örnek olabilir. Bazı durumlarda ise hiç örnek ipucu sağlanmayacaktır. Yönlendirici, örnek ipucunun varlığını kontrol eder ve bu ipucunun yönlendirme davranışını değiştirmek için kullanılıp kullanılmayacağına karar verir.

Bir `örnek` ipucu varsa, bu ipucunun alındığı veritabanı *instance._state.db* dosyasına kaydedilir. Yönlendiriciler, Django'nun ilgili örnekleri aynı veritabanından alma varsayılan davranışına uymak için sabit bir takma ada başvurmadan önce bu özniteliği kontrol etmelidir.

Yönlendiricileri kullanma

Veritabanı yönlendiricileri, *DATABASE_ROUTERS* ayarı kullanılarak kurulur. Bu ayar, her biri temel yönlendirici (`django.db.router`) tarafından kullanılması gereken bir yönlendiriciyi belirten sınıf adlarının bir listesini tanımlar.

Temel yönlendirici, Django'nun veritabanı işlemleri tarafından veritabanı kullanımını belirlemek için kullanılır. Bir sorgu hangi veritabanının kullanılacağını belirlemesi gerektiğinde, bir model ve (varsa) bir ipucu sağlayarak temel yönlendiriciyi çağırır. Temel yönlendirici, bir tanesi veritabanı önerisi döndürene kadar her yönlendirici sınıfını sırayla dener. Hiçbir yönlendirici öneri döndürmezse, temel yönlendirici ipucu örneğinin *instance._state.db* dosyasını dener. İpucu örneği sağlanmamışsa veya *instance._state.db* `None` ise, temel yönlendirici `varsayılan` veritabanını tahsis eder.

Bir örnek

> **Sadece örnek amaçlıdır!**
>
> Bu örnek, yönlendirici altyapısının veritabanı kullanımını nasıl değiştirebileceğini göstermek amacıyla hazırlanmıştır. Yönlendiricilerin nasıl kullanıldığını göstermek için bazı karmaşık konular kasıtlı olarak göz ardı edilmiştir.

`myapp` içindeki modellerden herhangi biri `diğer` veritabanı dışındaki modellerle ilişkiler içeriyorsa, bu örnek çalışmayacaktır. Veritabanları arası ilişkiler, Django'nun şu anda işleyemediği referans bütünlüğü sorunlarına yol açar.

Açıklanan birincil/çoğaltma (bazı veritabanlarında ana/yardımcı olarak adlandırılır) yapılandırması da kusurludur – çoğaltma gecikmesini (yani, bir yazma işleminin çoğaltmalara yayılması için geçen süre nedeniyle ortaya çıkan sorgu tutarsızlıklarını) ele almak için herhangi bir çözüm sunmaz. Ayrıca, işlemlerin veritabanı kullanım stratejisiyle etkileşimini de dikkate almaz.

Peki, bu pratikte ne anlama geliyor? Başka bir örnek yapılandırmayı ele alalım. Bu yapılandırmada birkaç veritabanı olacaktır: biri `kimlik doğrulama` uygulaması için, diğer tüm uygulamalar ise iki okuma replikası içeren birincil/replika kurulumunu kullanacaktır. İşte bu veritabanlarını belirten ayarlar:

```
DATABASES = {
    "default": {},
    "auth_db": {
        "NAME": "auth_db_name",
        "ENGINE": "django.db.backends.mysql", "USER":
        "mysql_user",
        "PASSWORD": "swordfish",
    },
    "primary": {
        "NAME": "primary_name",
        "ENGINE": "django.db.backends.mysql", "USER":
        "mysql_user",
        "PASSWORD": "spam",
    },
    "replica1": {
        "NAME": "replica1_name",
        "ENGINE": "django.db.backends.mysql", "USER":
        "mysql_user",
        "PASSWORD": "eggs",
    },
    "replica2": {
        "NAME": "replica2_name",
        "ENGINE": "django.db.backends.mysql
        ", "USER": "mysql_user",
        "PASSWORD": "bacon",
    },
}
```

Şimdi yönlendirmeyi halletmemiz gerekiyor. Öncelikle, `kimlik doğrulama` ve

`içerik türü` uygulamaları için sorguları `auth_db`'ye gönderebilen bir yönlendiriciye ihtiyacımız var (`otentikasyon` modelleri `ContentType` ile bağlantılı olduğundan, aynı veritabanında depolanmaları gerekir):

```
class AuthRouter:
    """
     auth ve contenttypes uygulamalarındaki modellere ait
      tüm veritabanı işlemlerini kontrol eden bir router.
    """

    route_app_labels = {"auth", "contenttypes"}

    def db_for_read(self, model, **hints):
        """
            auth ve contenttypes modellerini okuma girişimleri auth_db'
        ye yönlendirilir.
         """
        if model._meta.app_label in self.route_app_labels:
            return "auth_db"
        return None

    def db_for_write(self, model, **hints):
        """
            auth ve contenttypes modellerini yazma girişimleri auth_db'
        ye yönlendirilir.
        . """
        if model._meta.app_label in self.route_app_labels:
            return "auth_db"
        return None

    def allow_relation(self, obj1, obj2, **hints):
        """
        auth veya contenttypes uygulamalarındaki bir model söz konusuysa
            ilişkilere izin verilir.
        """
        if (
            obj1._meta.app_label in self.route_app_labels
            or obj2._meta.app_label in self.route_app_labels
        ):
            return True
        return None

    def allow_migrate(self, db, app_label, model_name=None, **hints):
        """
```

(önceki sayfadan devam)

```
        auth ve contenttypes uygulamalarının yalnızca 'auth_db' veritabanında yer
        aldığından emin olun.
        """
        if app_label in self.route_app_labels:
            db == "auth_db" ise
        return None
```

Ayrıca, diğer tüm uygulamaları birincil/yedek yapılandırmasına yönlendiren ve okumak için rastgele bir yedek seçen bir yönlendirici de istiyoruz:

```
import random



class PrimaryReplicaRouter:
    def db_for_read(self, model, **hints):
        """
        Okuma işlemleri rastgele seçilen bir replica'ya yönlendirilir.
        """
        return random.choice(["replica1", "replica2"])

    def db_for_write(self, model, **hints):
        """
        Yazma işlemleri her zaman primary'e gönderilir.
        """
        return "primary"

    def allow_relation(self, obj1, obj2, **hints):
        """
        Her iki nesne de primary/replica havuzundaysa
         nesneler arasındaki ilişkilere izin verilir.
        """
        db_set = {"primary", "replica1", "replica2"}
        if obj1._state.db in db_set and obj2._state.db in db_set:
            return True
        return None

    def allow_migrate(self, db, app_label, model_name=None, **hints):
        """
        auth olmayan tüm modeller bu havuza girer.
        """
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
        return True
```

Son olarak, ayar dosyasında aşağıdakileri ekliyoruz (`path.to.` kısmını, yönlendiricilerin tanımlandığı modül(ler)e giden gerçek Python yoluyla değiştirerek):

```
DATABASE_ROUTERS = ["path.to.AuthRouter", "path.to.PrimaryReplicaRouter"]
```

Yönlendiricilerin işlenme sırası önemlidir. Yönlendiriciler, *DATABASE_ROUTERS* ayarında listelendikleri sırayla sorgulanır. Bu örnekte, `AuthRouter`, `PrimaryReplicaRouter'dan` önce işlenir ve sonuç olarak, `auth` dizinindeki modellerle ilgili kararlar, diğer herhangi bir karar alınmadan önce işlenir. *DATABASE_ROUTERS* ayarında iki yönlendirici başka bir sırayla listelenmiş olsaydı, `PrimaryReplicaRouter.allow_migrate()` önce işlenirdi. PrimaryReplicaRouter uygulamasının her şeyi kapsayan yapısı, tüm modellerin tüm veritabanlarında kullanılabilir olacağı anlamına gelir.

Bu kurulum yüklendikten ve tüm veritabanları Veritabanlarınızı senkronize etme bölümüne göre taşındıktan sonra, biraz Django kodu çalıştıralım:

```
>>> # Bu arama 'auth_db' veritabanında gerçekleştirilecektir
>>> fred = User.objects.get(username="fred")
>>> fred.first_name = "Frederick"

>>> # Bu kaydetme işlemi de 'auth_db' veritabanına yönlendirilecektir
>>> fred.save()

>>> # Bu alma işlemi rastgele bir replika veritabanına tahsis edilecektir
>>> dna = Person.objects.get(name="Douglas Adams")

>>> # Yeni bir nesne oluşturulduğunda veritabanında bir tahsisatı yoktur
>>> mh = Book(title="Mostly Harmless")

>>> # Bu atama yönlendiriciye danışacak ve mh'yi
>>> # yazar nesnesiyle aynı veritabanına
>>> mh.author = dna

>>> # Bu kaydetme işlemi, 'mh' örneğini birincil veritabanına zorla kaydedecektir...
>>> mh.save()

>>> # ... ancak nesneyi yeniden alırsak, bir kopyada geri dönecektir
>>> mh = Book.objects.get(title="Mostly Harmless")
```

Bu örnekte, auth uygulamasındaki modellerle etkileşimi yönetmek üzere bir yönlendirici ve diğer tüm uygulamalarla etkileşimi yönetmek üzere başka yönlendiriciler tanımlanmıştır. `Varsayılan` veritabanınızı boş bıraktıysanız ve

, geçiş yapmadan önce yönlendiricilerinizin *INSTALLED_APPS* içindeki tüm uygulamaların adlarını yönetmesi gerekir. Tek bir veritabanında bir arada olması gereken katkı uygulamaları hakkında bilgi için katkı uygulamalarının davranışı bölümüne bakın.

Veritabanını manuel olarak seçme

Django ayrıca, kodunuzdaki veritabanı kullanımı üzerinde tam kontrol sahibi olmanızı sağlayan bir API sunar. Manuel olarak belirtilen bir veritabanı tahsisi, bir yönlendirici tarafından tahsis edilen veritabanına göre öncelikli olacaktır.

`QuerySet` için veritabanını manuel olarak seçme

`QuerySet` "zincirinin" herhangi bir noktasında bir `QuerySet` için veritabanını seçebilirsiniz.
, belirtilen veritabanını kullanan başka bir `QuerySet` elde etmek için `kullanın`.

using`()` tek bir argüman alır: sorguyu çalıştırmak istediğiniz veritabanının takma adı. Örneğin:

```
>>> # Bu, 'default' veritabanında çalışacaktır.
>>> Author.objects.all()

>>> # Bu da öyle.
>>> Author.objects.using("default")

>>> # Bu kod, 'other' veritabanında çalışacaktır.
>>> Author.objects.using("other")
```

`save()` için veritabanı seçme

Verilerin hangi veritabanına kaydedileceğini belirtmek için `Model.save()` işlevinde `using` anahtar sözcüğünü

kullanın. Örneğin, bir nesneyi `legacy_users` veritabanına kaydetmek için şu komutu kullanabilirsiniz:

```
>>> my_object.save(using="legacy_users")
```

`using` belirtmezseniz, `save()` yöntemi yönlendiriciler tarafından ayrılan varsayılan veritabanına kaydeder.

Bir nesneyi bir veritabanından diğerine taşıma

Bir örneği bir veritabanına kaydettiyseniz, bu örneği yeni bir veritabanına taşımak için `save(using=...)` işlevini kullanmak cazip gelebilir. Ancak, gerekli önlemleri almazsanız, bu durum bazı beklenmedik sonuçlara yol açabilir.

Aşağıdaki örneği ele alalım:

```
>>> p = Person(name="Fred")
>>> p.save(using="first")    # (ifade 1)
>>> p.save(using="second")    # (ifade 2)
```

1. ifadede, yeni bir `Person` nesnesi `ilk` veritabanına kaydedilir. Bu aşamada `p'nin` bir birincil anahtarı bulunmadığından, Django bir SQL `INSERT` ifadesi yürütür. Bu işlem bir birincil anahtar oluşturur ve Django bu birincil anahtarı `p'ye` atar.

2. ifadede kaydetme işlemi gerçekleştiğinde, `p'nin` zaten bir birincil anahtar değeri vardır ve Django bu birincil anahtarı yeni veritabanında kullanmaya çalışır. Birincil anahtar değeri `ikinci` veritabanında kullanılmıyorsa, herhangi bir sorun yaşamazsınız; nesne yeni veritabanına kopyalanır.

Ancak, p'nin birincil anahtarı `ikinci` veritabanında zaten kullanılıyorsa, p kaydedildiğinde `ikinci` veritabanındaki mevcut nesne, `p` k a y d e d ildiğinde geçersiz kılınacaktır.

Bunu iki şekilde önleyebilirsiniz. İlk olarak, nesnenin birincil anahtarını silebilirsiniz. Bir nesnenin birincil anahtarı yoksa, Django onu yeni bir nesne olarak değerlendirir ve `ikinci` veritabanında herhangi bir veri kaybını önler:

```
>>> p = Person(name="Fred")
>>> p.save(using="first")
>>> p.pk = None # Birincil anahtarı temizleyin.
>>> p.save(using="second") # Tamamen yeni bir nesne yazın.
```

İkinci seçenek, Django'nun bir SQL `INSERT` işlemi yapmasını sağlamak için `save()` işlevine `force_insert` seçeneğini kullanmaktır:

```
>>> p = Person(name="Fred")
>>> p.save(using="first")
>>> p.save(using="second", force_insert=True)
```

Bu, `Fred` adlı kişinin her iki veritabanında da aynı birincil anahtara sahip olmasını sağlar. `İkinci` veritabanına kaydetmeye çalıştığınızda bu birincil anahtar zaten kullanılıyorsa, bir hata oluşur.

Silinecek veritabanını seçme

Varsayılan olarak, mevcut bir nesneyi silme çağrısı, o nesnenin ilk olarak alınmasında kullanılan veritabanında gerçekleştirilir:

```
>>> u = User.objects.using("legacy_users").get(username="fred")
>>> u.delete() # `legacy_users` veritabanından silinecektir
```

Bir modelin silineceği veritabanını belirtmek için, `Model.delete()` yöntemine bir `using` anahtar sözcüğü argümanı geçirin. Bu argüman, `save()` yöntemindeki `using` anahtar sözcüğü argümanı ile aynı şekilde çalışır.

Örneğin, bir kullanıcıyı `legacy_users` veritabanından `new_users` veritabanına taşıyorsanız, şu komutları kullanabilirsiniz:

```
>>> user_obj.save(using="new_users")
>>> user_obj.delete(using="legacy_users")
```

Birden fazla veritabanıyla yöneticileri kullanma

Yöneticilere varsayılan olmayan bir veritabanına erişim izni vermek için yöneticiler üzerinde `db_manager()` yöntemini kullanın.

Örneğin, veritabanına erişen özel bir yönetici yönteminizi olduğunu varsayalım – `User.objects.create_user()`. `create_user()` bir `QuerySet` yöntemi değil, bir yönetici yöntemi olduğu için `User.objects.using('new_users').create_user()` yapamazsınız. (`create_user()` yöntemi yalnızca yönetici olan `User.objects` üzerinde kullanılabilir, yöneticiden türetilen `QuerySet` nesnelerinde kullanılamaz.) Çözüm, `db_manager()`'ı şu şekilde kullanmaktır:

```
User.objects.db_manager("new_users").create_user(...)
```

`db_manager()`, belirttiğiniz veritabanına bağlı yöneticinin bir kopyasını döndürür.

Birden fazla veritabanında `get_queryset()` kullanımı

Yöneticinizde `get_queryset()` yöntemini geçersiz kılıyorsanız, ya üst sınıfta bu yöntemi çağırdığınızdan (`super()` kullanarak) emin olun ya da yöneticideki `_db` özniteliğini (kullanılacak veritabanının adını içeren bir dize) uygun şekilde işlediğinizden emin olun.

Örneğin, get_queryset yönteminden özel bir `QuerySet` sınıfı döndürmek istiyorsanız, bunu şu şekilde yapabilirsiniz:

```
class MyManager(models.Manager): def
    get_queryset(self):
        qs = CustomQuerySet(self.model) if
        self._db is not None:
            qs = qs.using(self._db)
        return qs
```

Django'nun yönetici arayüzünde birden fazla veritabanını gösterme

Django'nun admin arayüzü, birden fazla veritabanı için açık bir destek sunmaz. Yönlendirici zincirinizde belirtilen veritabanından farklı bir veritabanındaki bir model için admin arayüzü sağlamak istiyorsanız, admin arayüzünü içerik için belirli bir veritabanını kullanmaya yönlendirecek özel *ModelAdmin* sınıfları yazmanız gerekir.

`ModelAdmin` nesneleri, çoklu veritabanı desteği için özelleştirme gerektiren aşağıdaki yöntemlere sahiptir:

```
class MultiDBModelAdmin(admin.ModelAdmin):
                    # Alternatif veritabanının adı için kullanışlı bir sabit.
  using = "other"

  def save_model(self, request, obj, form, change):
      # Django'ya nesneleri 'other' veritabanına kaydetmesini söyler.
        obj.save(using=self.using)
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
    def delete_model(self, request, obj):
        # Django'ya nesneleri 'other' veritabanından silmesini söyleyin
            obj.delete(using=self.using)

        def get_queryset(self, request):
        # Django'ya 'other' veritabanında nesneleri aramasını söyle.
        return super().get_queryset(request).using(self.using)

    def formfield_for_foreignkey(self, db_field, request, **kwargs):
        # Django'ya, ForeignKey widget'larını 'diğer' veritabanındaki bir
        sorgu kullanarak doldurmasını söyle.
        return super().formfield_for_foreignkey(
            db_field, request, using=self.using, **kwargs
        )

    def formfield_for_manytomany(self, db_field, request, **kwargs):
        # Django'ya, 'diğer' veritabanındaki bir sorguyu kullanarak
        ManyToMany widget'larını doldurmasını söyleyin.
        return super().formfield_for_manytomany(
            db_field, request, using=self.using, **kwargs
        )
```

Burada sunulan uygulama, belirli bir türdeki tüm nesnelerin belirli bir veritabanında depolandığı (örneğin, tüm `User` nesneleri `diğer` veritabanında bulunur) çoklu veritabanı stratejisini hayata geçirir. Çoklu veritabanı kullanımınız daha karmaşıksa, `ModelAdmin'inizin` bu stratejiyi yansıtması gerekecektir.

InlineModelAdmin nesneleri de benzer şekilde işlenebilir. Bunlar için üç özelleştirilmiş yöntem gereklidir:

```
class MultiDBTabularInline(admin.TabularInline): using =
    "other"

    def get_queryset(self, request):
        # Django'ya 'other' veritabanında inline nesneleri aramasını söyleyin.
        return super().get_queryset(request).using(self.using)

    def formfield_for_foreignkey(self, db_field, request, **kwargs):
        # Django'ya, 'diğer' veritabanındaki bir sorguyu kullanarak
        ForeignKey widget'larını doldurmasını söyle.
        return super().formfield_for_foreignkey(
            db_field, request, using=self.using, **kwargs
```

(sonraki sayfada devam ediyor)

(önceki sayfadan devam)

```
        )

    def formfield_for_manytomany(self, db_field, request, **kwargs):
        # Django'ya, 'diğer' veritabanındaki bir sorguyu kullanarak
        ManyToMany widget'larını doldurmasını söyle.
        return super().formfield_for_manytomany(
            db_field, request, using=self.using, **kwargs
        )
```

Model yönetici tanımlarınızı yazdıktan sonra, bunları herhangi bir `Admin` örneğine kaydedebilirsiniz:

```
from django.contrib import admin
from myapp.models import Author, Book, Publisher

# Tanımlandıkları yerden özel ModelAdmin ve TabularInline'ımızı içe aktarın.
from myproject.admin import MultiDBModelAdmin, MultiDBTabularInline


# Belirli modellerle kullanılmak üzere çoklu veritabanı yönetim nesnelerini özelleştirin.
class BookInline(MultiDBTabularInline): model =
    Book


class PublisherAdmin(MultiDBModelAdmin): inlines
    = [BookInline]


admin.site.register(Author, MultiDBModelAdmin)
admin.site.register(Publisher, PublisherAdmin)

othersite = admin.AdminSite("othersite") othersite.register(Publisher,
MultiDBModelAdmin)
```

Bu örnek, iki yönetici sitesi kurar. İlk sitede, `Author` ve `Publisher` nesneleri gösterilir; `Publisher` nesneleri, o yayıncı tarafından yayınlanan kitapları gösteren bir tablo içerir. İkinci site, iç satırlar olmadan sadece yayıncıları gösterir.

Birden fazla veritabanıyla ham imleçleri kullanma

Birden fazla veritabanı kullanıyorsanız, belirli bir veritabanına ait bağlantıyı (ve imleci) almak için `django.db.connections`'ı kullanabilirsiniz. `django.db.connections`, takma adını kullanarak belirli bir bağlantıyı almanızı sağlayan sözlük benzeri bir nesnedir:

```
from django.db import connections

with connections["my_db_alias"].cursor() as cursor:
    ...
```

Birden fazla veritabanının sınırlamaları Veritabanları arası ilişkiler

Django şu anda birden fazla veritabanını kapsayan yabancı anahtar veya çok-çok ilişkileri için herhangi bir destek sağlamamaktadır. Modelleri farklı veritabanlarına bölmek için bir yönlendirici kullandıysanız, bu modeller tarafından tanımlanan tüm yabancı anahtar ve çok-çok ilişkileri tek bir veritabanının içinde olmalıdır.

Bunun nedeni referans bütünlüğüdür. İki nesne arasındaki ilişkiyi sürdürebilmek için Django'nun, ilişkili nesnenin birincil anahtarının geçerli olduğunu bilmesi gerekir. Birincil anahtar ayrı bir veritabanında depolanıyorsa, bu anahtarın geçerliliğini kolayca değerlendirmek mümkün olmaz.

Postgres, SQLite, Oracle veya InnoDB ile MySQL kullanıyorsanız, bu durum veritabanı bütünlüğü düzeyinde uygulanır – veritabanı düzeyindeki anahtar kısıtlamaları, doğrulanamayan ilişkilerin oluşturulmasını engeller.

Ancak, MyISAM tablolarıyla MySQL kullanıyorsanız, referans bütünlüğü zorunlu değildir; bu nedenle, veritabanları arası yabancı anahtarları "taklit" edebilirsiniz. Ancak bu yapılandırma, Django tarafından resmi olarak desteklenmemektedir.

Contrib uygulamalarının davranışı

Birkaç contrib uygulaması modeller içerir ve bazı uygulamalar diğerlerine bağımlıdır. Veritabanları arası ilişkiler mümkün olmadığından, bu durum bu modelleri veritabanları arasında nasıl bölebileceğiniz konusunda bazı kısıtlamalar yaratır:

- Uygun bir yönlendirici kullanıldığında, `contenttypes.ContentType`, `sessions.Session` ve `sites.Site` nesnelerinin her biri herhangi bir veritabanında depolanabilir.
- `auth` modelleri — `User`, `Group` ve `Permission` — birbirleriyle ve `ContentType` ile bağlantılıdır, bu nedenle `ContentType` ile aynı veritabanında depolanmaları gerekir.
- `admin`, `auth'a` bağlıdır; bu nedenle modelleri `auth` ile aynı veritabanında olmalıdır.
- `flatpages` ve `redirects`, `sites'e` bağlıdır; bu nedenle modelleri `sites` ile aynı veritabanında bulunmalıdır.

Buna ek olarak, bazı nesneler, *migrate'in* bunları veritabanında tutmak için bir tablo oluşturmasının hemen ardından otomatik olarak oluşturulur:

- varsayılan bir `Site`,

- her model için bir `ContentType` (o veritabanında depolanmayanlar dahil),
- her model için İzinler (o veritabanında depolanmayanlar dahil).

Birden fazla veritabanının bulunduğu yaygın kurulumlarda, bu nesnelerin birden fazla veritabanında bulunması yararlı değildir. Yaygın kurulumlar arasında birincil/yedek ve harici veritabanlarına bağlanma yer alır. Bu nedenle, bu üç modeli yalnızca tek bir veritabanıyla senkronize etmeyi sağlayan bir veritabanı yönlendiricisi yazmanız önerilir. Tablolarının birden fazla veritabanında bulunmasına gerek olmayan katkı ve üçüncü taraf uygulamalar için de aynı yaklaşımı kullanın.

> **Uyarı**
>
> İçerik türlerini birden fazla veritabanına senkronize ediyorsanız, bu içerik türlerinin birincil anahtarlarının veritabanları arasında uyuşmayabileceğini unutmayın. Bu durum veri bozulmasına veya veri kaybına yol açabilir.

3.2.9 Tablo alanları

Veritabanı sistemlerinde performansı optimize etmek için yaygın olarak kullanılan bir yöntem, disk düzenini organize etmek için tablo alanlarının kullanılmasıdır.

> **Uyarı**
>
> Django sizin için tablo alanları oluşturmaz. Tablo alanlarının oluşturulması ve yönetilmesi ile ilgili ayrıntılar için lütfen veritabanı motorunuzun belgelerine bakın.

Tablolar için tablo alanları bildirimi

Bir model tarafından oluşturulan tablo için, modelin `Meta sınıfı` içinde *db_tablespace* seçeneği sağlanarak bir tablo alanı belirtilebilir. Bu seçenek, modeldeki ManyToManyFields için otomatik olarak oluşturulan tabloları da etkiler.

DEFAULT_TABLESPACE ayarını kullanarak *db_tablespace* için bir varsayılan değer belirleyebilirsiniz. Bu, yerleşik Django uygulamaları ve kodunu kontrol edemediğiniz diğer uygulamalar için bir tablo alanı ayarlamak amacıyla kullanışlıdır.

Dizinler için tablo alanları bildirimi

Dizin oluşturucusuna *db_tablespace* seçeneğini geçirerek, dizin için kullanılacak tablo alanının adını belirtebilirsiniz. Tek alanlı dizinler için, `alan` oluşturucusuna *db_tablespace* seçeneğini geçirerek, alanın sütun dizini için alternatif bir tablo alanı belirtebilirsiniz. Sütunda dizin yoksa, bu seçenek göz ardı edilir.

DEFAULT_INDEX_TABLESPACE ayarını kullanarak *db_tablespace* için bir varsayılan değer belirtebilirsiniz.

db_tablespace belirtilmezse ve *DEFAULT_INDEX_TABLESPACE* ayarlanmamışsa, dizin tablolarla aynı tablo alanında oluşturulur.

Bir örnek

```
class TablespaceExample(models.Model):
    name = models.CharField(max_length=30, db_index=True, db_tablespace="indexes") data =
    models.CharField(max_length=255, db_index=True)
    shortcut = models.CharField(max_length=7)
    edges = models.ManyToManyField(to="self", db_tablespace="indexes")

    sınıf Meta:
        db_tablespace = "tables"
        indexes = [models.Index(fields=["shortcut"], db_tablespace="other_indexes")]
```

Bu örnekte, `TablespaceExample` modeli tarafından oluşturulan tablolar (yani model tablosu ve çok-çok tablosu) `tables` tablespace'inde depolanacaktır. name alanı için dizin ve çok-çok tablosundaki dizinler, `indexes` tablo alanında depolanır. `data` alanı da bir dizin oluşturur, ancak bunun için herhangi bir tablo alanı belirtilmediğinden, varsayılan olarak model tablo `alanı tables'da` depolanır. `shortcut` alanı için dizin, `other_indexes` tablo alanında depolanır.

Veritabanı desteği

PostgreSQL ve Oracle tablespace'leri destekler. SQLite, MariaDB ve MySQL desteklemez.

Tablespace desteği olmayan bir backend kullandığınızda, Django tablespace ile ilgili tüm option'ları ignore eder.

Sorular:

Django'nun varsayılan transaction davranışı nedir?

Cevap: Django varsayılan olarak autocommit modunda çalışır .

on_commit() callback'i ne zaman çalışır?

A) Her zaman çalışır
B) Transaction rollback edilince çalışır
C) Transaction başarıyla commit edilince çalışır
D) View çağrıldığında çalışır

Cevap : C Transaction rollback edilirse callback hiç çalışmaz, discard edilir.